

Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №5

По дисциплине: «Аппаратно-программное обеспечение ЭВМ и сетей»

Тема: «Анализ сетевого трафика и протоколов (на базе WireShark)»

Выполнила:

Студент 4 курса

Группы АС-66

Езепчук А.С.

Проверила:

Кулик А.Д.

Цель работы: приобретение навыков анализа сетевого трафика компьютерных сетей; изучение структуры сетевых протоколов различных уровней

Вариант 6

Постановка задачи. Запустив Wireshark на захват, выполнить загрузку страницы согласно варианту

6	https://ria.ru
---	---

Остановить и сохранить

захват. Для захваченных пакетов определить статистические данные:

- процентное соотношение трафика разных протоколов в сети;
- среднюю скорость кадров/сек;
- среднюю скорость байт/сек;
- минимальный, максимальный и средний размеры пакета;
- степень использования полосы пропускания канала (загрузку сети).

Отфильтровать в захвате IP пакеты. Определить статистические данные:

- процентное соотношение трафика разных протоколов стека tcp/ip в сети;
- средний, минимальный, максимальный размеры пакета.

На примере третьего захваченного IP-пакета указать структуры протокола канального уровня (протокола Ethernet 802.3, Wi-Fi 802.11, либо другого, используемого в вашей конфигурации) и протокола IPv4. Отметить поля заголовков и описать их и интерпретировать их значения.

Запустив Wireshark на захват, выполнить команду ping для IP адреса компьютера (предварительно определив его адрес с помощью ipconfig; пример команды: ping 172.17.20.246). Сохранить результат. Сформировав нужный фильтр, отфильтровать пакеты, относящиеся к выполнению команды ping. На базе полученных пакетов и значений их полей интерпретировать результат работы утилиты ping. Описать все протоколы, используемые утилитой. Сформировать не менее 3-х сложных фильтров захвата с использованием полей протоколов, операторов сравнения (таблицы 1 и 2 из файла теоретические_указания4) и логических операторов; каждый раз перезапуская захват, для каждого фильтра захватить соответствующие пакеты.

Выполнить анализ ARP-протокола по примеру из методических указаний.

Выполнить анализ TCP-сеансов по примеру из методических указаний

Оформить отчет, содержащий титульный лист, данные варианта задания, скриншоты полученных результатов

Ход работы:

Начинаем захват и открываем в браузере страницу сайта ria.ru. Ждем 15 секунд чтобы сайт успел прогрузиться и отправить пакеты. После этого останавливаем захват. После остановки захвата сохраняем как файл lab5_first.pcapng.

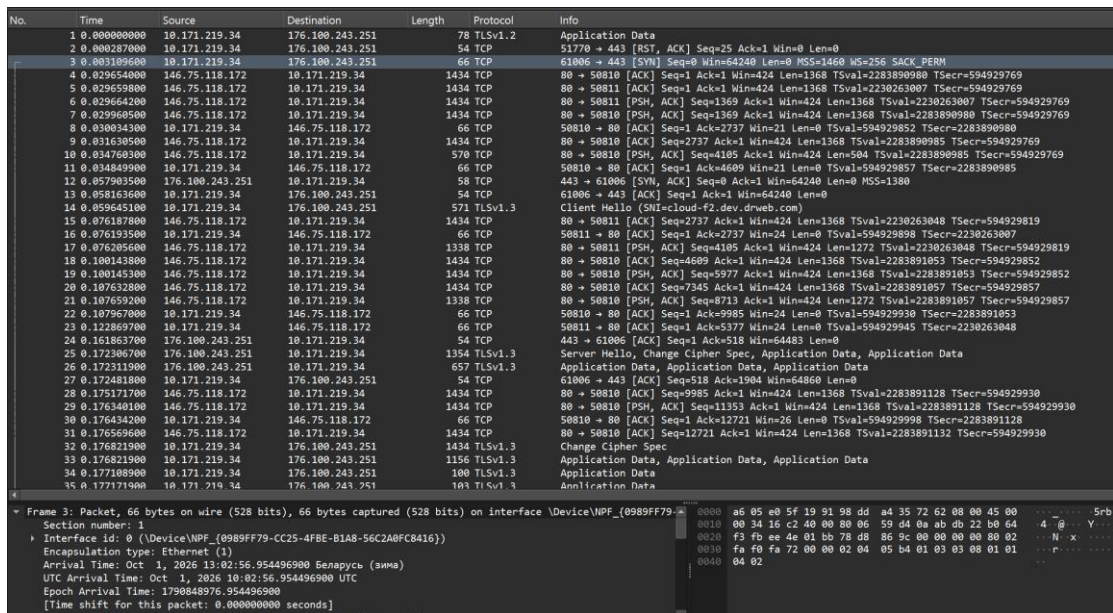


Рис. 1 - Интерфейс Wireshark

Для определения процентное соотношение трафика разных протоколов в сети используем вкладку Статистики Иерархия протоколов (Рис. 2)

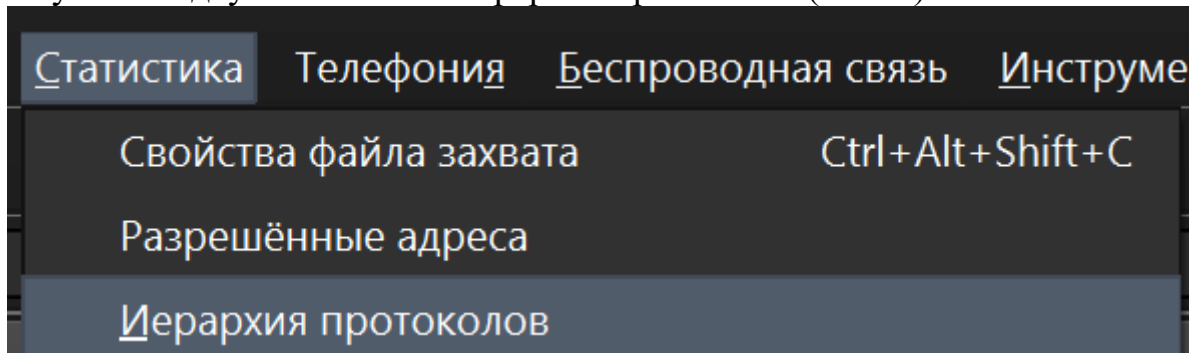


Рис. 2 – Где искать иерархию

Рассматривая процентаж из Иерархии протоколов (Рис. 3) можно выделить, что большая часть приходится на ipv4 (99.8), а оставшиеся пару сотых процента приходятся на ipv6.

Протокол	Процент пакетов	Пакеты	Процент байтов	Байты	Бит/с	Конечные пакеты	Конечные байты	Конечные бит/с	PDU
Frame	100.0	3741	100.0	2415163	843 k	0	0	0	3741
Ethernet	100.0	3741	2.2	52374	18 k	0	0	0	3741
Internet Protocol Version 6	0.2	6	0.0	240	83	0	0	0	6
User Datagram Protocol	0.2	6	0.0	48	16	0	0	0	6
Multicast Domain Name System	0.2	6	0.0	522	182	6	522	182	6
Internet Protocol Version 4	99.8	3733	3.1	74660	26 k	0	0	0	3733
User Datagram Protocol	6.0	226	0.1	1808	631	0	0	0	226
QUIC IETF	2.7	100	2.6	61688	21 k	100	55241	19 k	117
Multicast Domain Name System	0.2	6	0.0	522	182	6	522	182	6
Domain Name System	3.2	120	0.4	9039	3158	120	9039	3158	120
Transmission Control Protocol	93.7	3507	3.8	92268	32 k	2779	77708	27 k	3507
Transport Layer Security	19.5	728	40.1	969248	338 k	728	915758	320 k	746
Address Resolution Protocol	0.1	2	0.0	56	19	2	56	19	2

Рис. 3 – Иерархия протоколов

Для получения статистических данных данного файла используем Свойства файла захвата (Рис. 4)

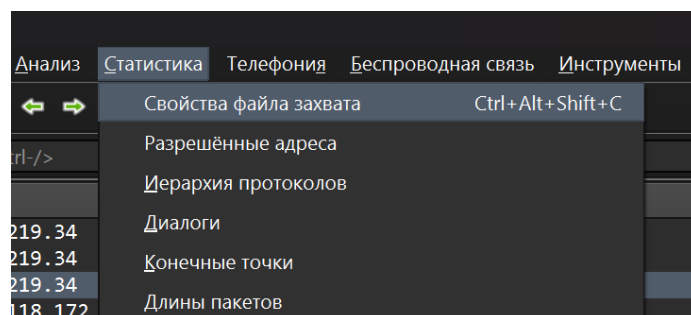


Рис. 4 – Где искать стату

Перейдя к окну (Рис. 5) мы получаем информацию о открытом файле захвата.

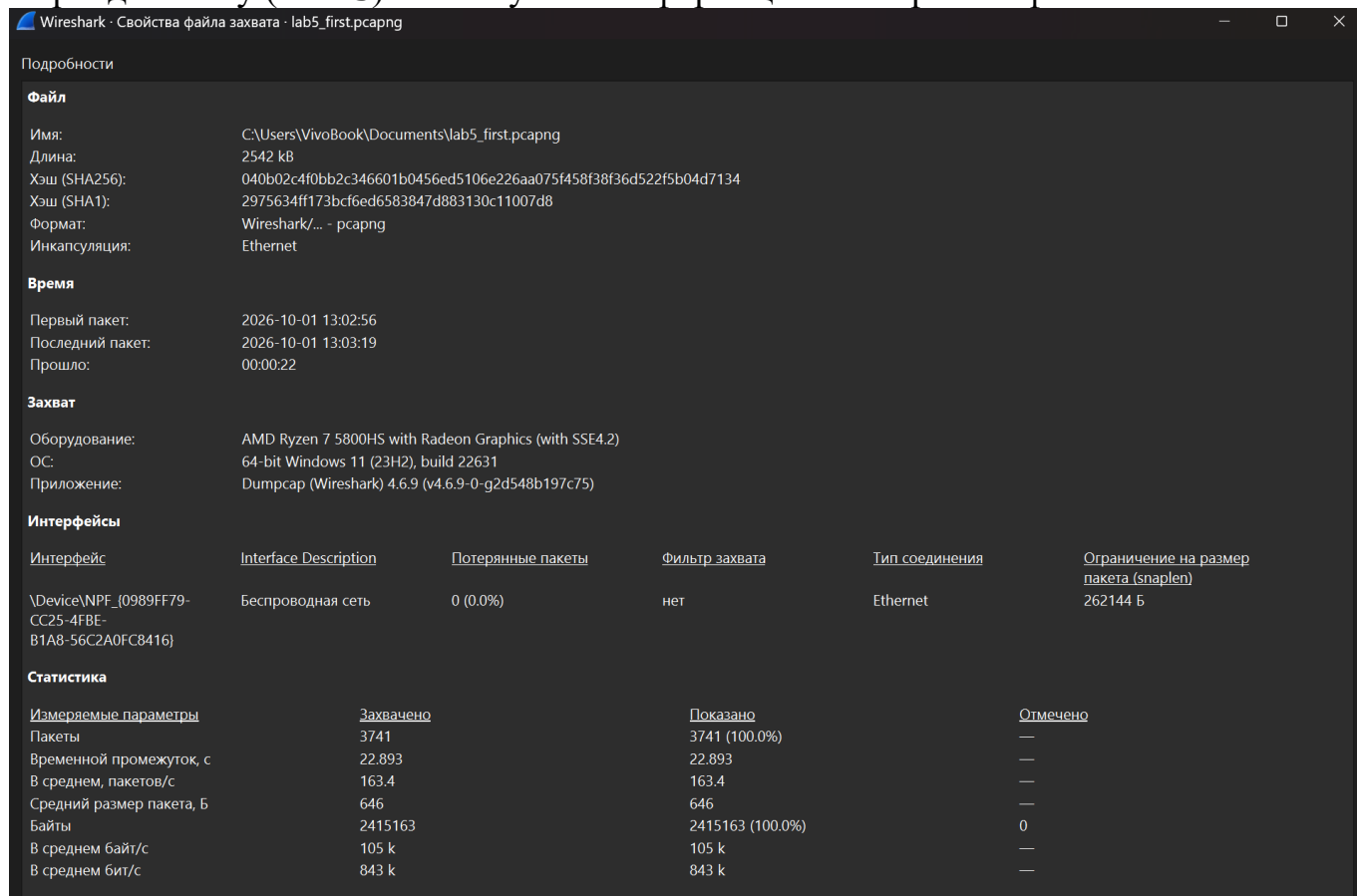


Рис. 5 – Свойства файла захвата

Для определения минимального и максимального размера пакетов используем пункт Длины пакетов (Рис. 6)

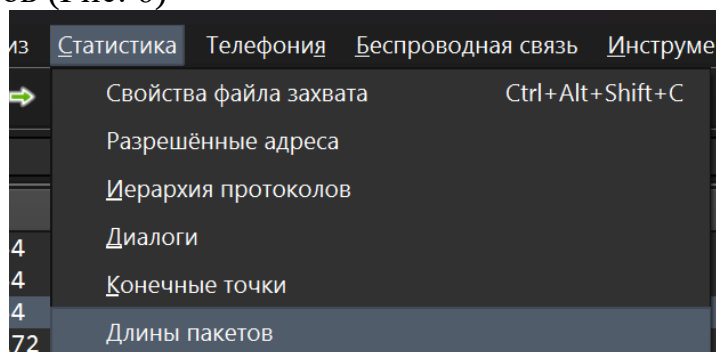


Рис. 6 – Где искать длину

Открывшееся окно (Рис. 7) предоставляет необходимую информацию.

Topic / Item	Cou	Average	Min Val	Max Val	Rate (ms)	Percent	Burst Rate	Burst Start
▼ Packet Lengths	3741	645,59	42	1434	0,1634	100%	2,0300	14,996
0-19	0	-	-	-	0,0000	0,00%	-	-
20-39	0	-	-	-	0,0000	0,00%	-	-
40-79	1443	61,20	42	79	0,0630	38,57%	1,0800	14,530
80-159	366	106,90	81	158	0,0160	9,78%	0,4400	14,996
160-319	145	230,22	163	317	0,0063	3,88%	0,1400	10,884
320-639	161	476,35	325	639	0,0070	4,30%	0,1100	14,545
640-1279	381	1053,45	640	1279	0,0166	10,18%	0,3000	15,904
1280-2559	1245	1426,73	1289	1434	0,0544	33,28%	0,5400	14,978
2560-5119	0	-	-	-	0,0000	0,00%	-	-
5120 and greater	0	-	-	-	0,0000	0,00%	-	-

Рис. 7 – Длины пакетов

Для определения загрузки сети мы построим график через пункт Графики ввода/вывода (Рис. 8)

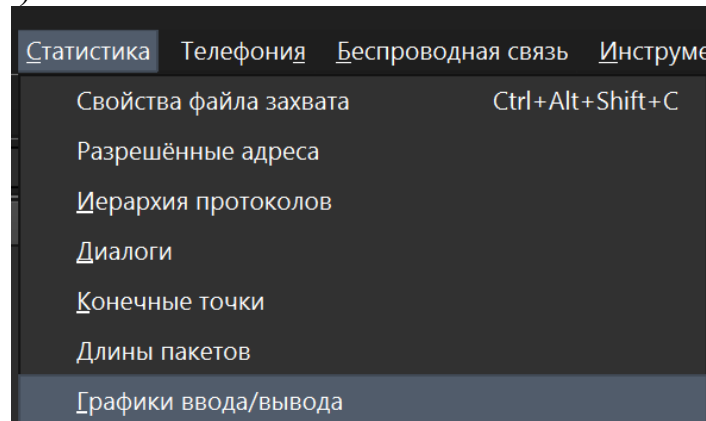


Рис. 8 – Где искать график

Строим график (Рис. 9), который показывает зависимость Бит от времени и получаем загрузку нашей сети в конкретный момент.

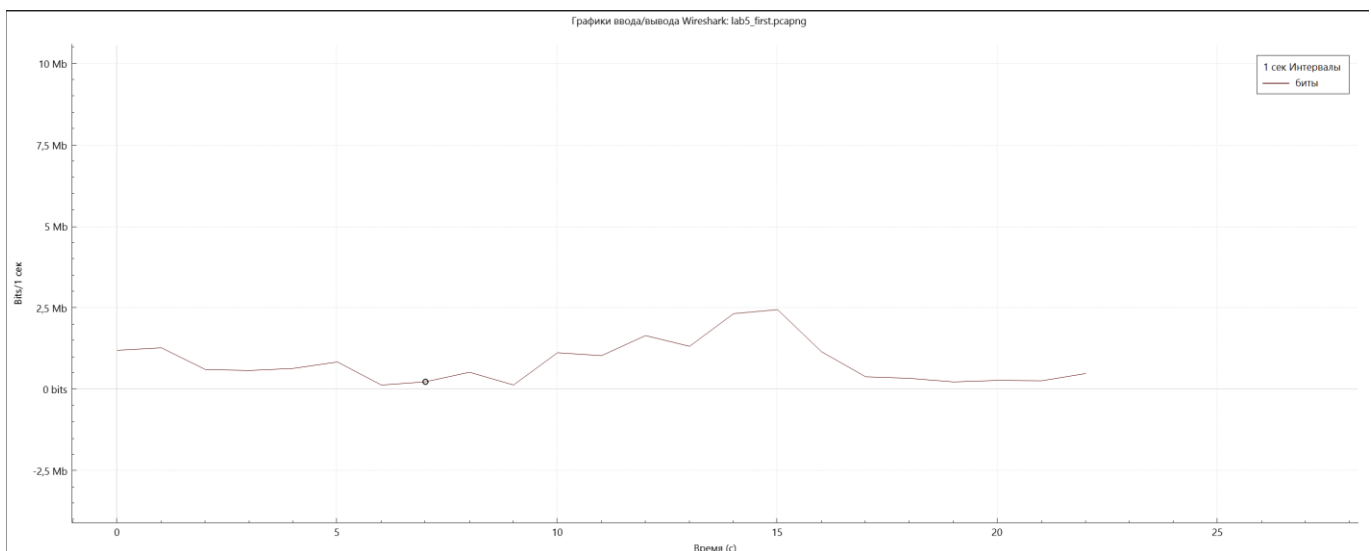


Рис. 9 – График Биты в секунду

Для более простой оценки отфильтрованных результатов используем ipv6, которых почти нет в общем пуле пакетов.

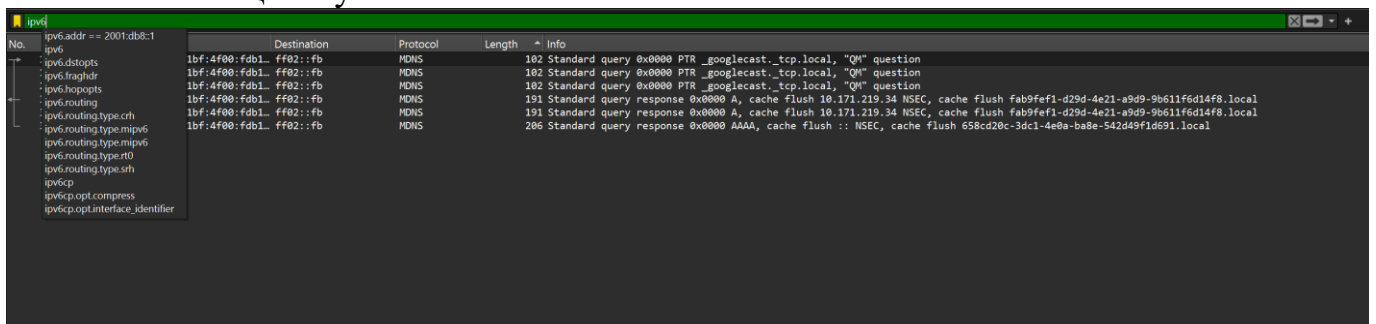
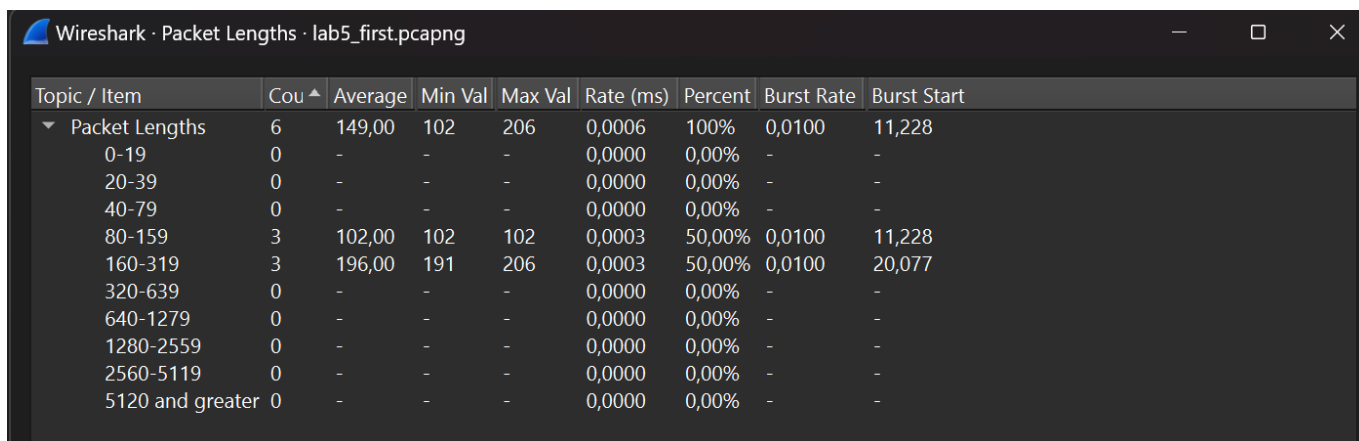


Рис. 10 – отфильтрованные ipv6

The image shows the 'Wireshark - Статистика иерархии протоколов - lab5_first.pcapng' window. The table displays the hierarchy of protocols and their respective statistics.

Протокол	Процент пакетов	Пакеты	Процент байтов	Байты	Бит/с	Конечные пакеты	Конечные байты	Конечные бит/с	PDU
Frame	100.0	6	100.0	894	659	0	0	0	6
Ethernet	100.0	6	9.4	84	61	0	0	0	6
Internet Protocol Version 6	100.0	6	26.8	240	176	0	0	0	6
User Datagram Protocol	100.0	6	5.4	48	35	0	0	0	6
Multicast Domain Name System	100.0	6	58.4	522	384	6	522	384	6

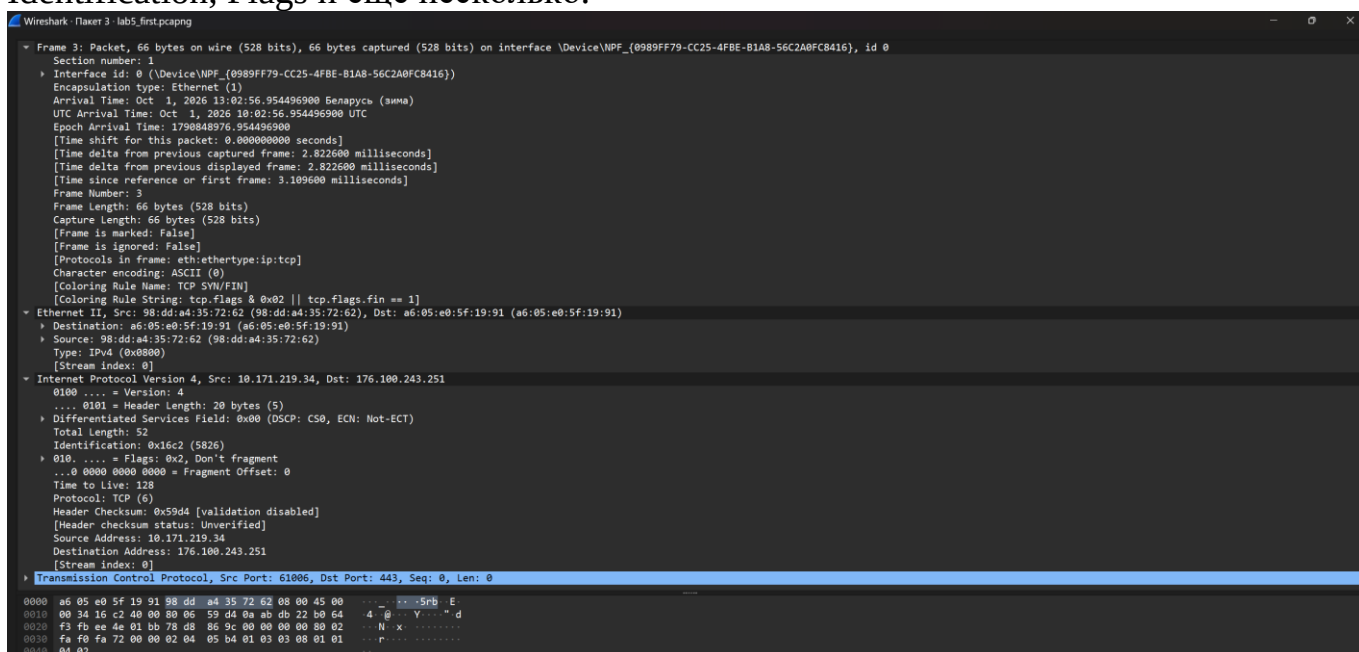
Рис. 11 – иерархия протоколов для ipv6



Topic / Item	Cou	Average	Min Val	Max Val	Rate (ms)	Percent	Burst Rate	Burst Start
Packet Lengths	6	149,00	102	206	0,0006	100%	0,0100	11,228
0-19	0	-	-	-	0,0000	0,00%	-	-
20-39	0	-	-	-	0,0000	0,00%	-	-
40-79	0	-	-	-	0,0000	0,00%	-	-
80-159	3	102,00	102	102	0,0003	50,00%	0,0100	11,228
160-319	3	196,00	191	206	0,0003	50,00%	0,0100	20,077
320-639	0	-	-	-	0,0000	0,00%	-	-
640-1279	0	-	-	-	0,0000	0,00%	-	-
1280-2559	0	-	-	-	0,0000	0,00%	-	-
2560-5119	0	-	-	-	0,0000	0,00%	-	-
5120 and greater	0	-	-	-	0,0000	0,00%	-	-

Рис. 12 – размер пакетов для ipv6

Нажав на третий пакет, переходим в окно с информацией о нем (Рис.13). Заголовок Ethernet имеет фиксированную длину 14 байт и содержит 3 поля – Destination (6 байт), Source (6 байт), Type (2 байта). Заголовок IPv4 минимально занимает 20 байт. Его основные поля это Version, Header Length, Differentiated Services, Total Length, Identification, Flags и ещё несколько.

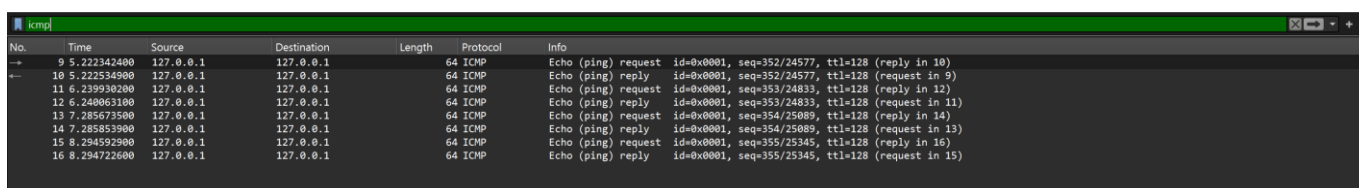


```

Frame 3: Packet, 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface \Device\NPF_{0989FF79-CC25-4FBE-B1A8-56C2A0FC8416}, id 0
Section number: 1
  Interface id: 0 (\Device\NPF_{0989FF79-CC25-4FBE-B1A8-56C2A0FC8416})
  Encapsulation type: Ethernet (1)
  Arrival Time: Oct  1, 2026 13:02:56.954496900 Беларусь (амма)
  UTC Arrival Time: Oct  1, 2026 10:02:56.954496900 UTC
  Epoch Arrival Time: 1790848976.954496900
  [Time shift for this packet: 0.000000000 seconds]
  [Time delta from previous captured frame: 2.822600 milliseconds]
  [Time delta from previous displayed frame: 2.822600 milliseconds]
  [Time since reference or first frame: 3.109600 milliseconds]
  Frame Number: 3
  Frame Length: 66 bytes (528 bits)
  Capture Length: 66 bytes (528 bits)
  [Frame is marked: False]
  [Frame is ignored: False]
  [Protocols in frame: eth:ethertype:ip:tcp]
  Character encoding: ASCII (0)
  [Coloring Rule Name: TCP SYN/FIN]
  [Coloring Rule String: tcp.flags & 0x02 || tcp.flags.fin == 1]
  Ethernet II, Src: 98:dd:a4:35:72:62 (98:dd:a4:35:72:62), Dst: a6:05:e0:5f:19:91 (a6:05:e0:5f:19:91)
    Destination: a6:05:e0:5f:19:91 (a6:05:e0:5f:19:91)
    Source: 98:dd:a4:35:72:62 (98:dd:a4:35:72:62)
    Type: IPv4 (0x0800)
    [Stream index: 0]
  Internet Protocol Version 4, Src: 10.171.219.34, Dst: 176.100.243.251
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 52
    Identification: 0x16c2 (5826)
    0100 .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 128
    Protocol: TCP (6)
    Header Checksum: 0x59d4 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 10.171.219.34
    Destination Address: 176.100.243.251
    [Stream index: 0]
  Transmission Control Protocol, Src Port: 61006, Dst Port: 443, Seq: 0, Len: 0
  
```

Рис. 13 – инфа по третьему пакету

Для пинга мы делаем отдельный файл захвата – lab5_second.pcapng. В консоли мы прописываем пинг и получаем результат (Рис.14). Утилита ping использует протоколы ICMP (Internet Control Message Protocol), IPv4 и Ethernet. У каждого пакета есть поле Type (8 для запроса, 0 для ответа). Идентификатор и номер позволяют сопоставить запрос с ответом.



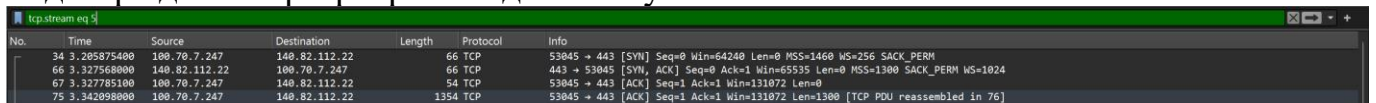
No.	Time	Source	Destination	Length	Protocol	Info
9	5.222342400	127.0.0.1	127.0.0.1	64	ICMP	Echo (ping) request id=0x0001, seq=352/24577, ttl=128 (reply in 10)
10	5.222534900	127.0.0.1	127.0.0.1	64	ICMP	Echo (ping) reply id=0x0001, seq=352/24577, ttl=128 (request in 9)
11	5.229930200	127.0.0.1	127.0.0.1	64	ICMP	Echo (ping) request id=0x0001, seq=353/24833, ttl=128 (reply in 12)
12	6.240863100	127.0.0.1	127.0.0.1	64	ICMP	Echo (ping) reply id=0x0001, seq=353/24833, ttl=128 (request in 11)
13	7.285673500	127.0.0.1	127.0.0.1	64	ICMP	Echo (ping) request id=0x0001, seq=354/25089, ttl=128 (reply in 14)
14	7.285853900	127.0.0.1	127.0.0.1	64	ICMP	Echo (ping) reply id=0x0001, seq=354/25089, ttl=128 (request in 13)
15	8.294592900	127.0.0.1	127.0.0.1	64	ICMP	Echo (ping) request id=0x0001, seq=355/25345, ttl=128 (reply in 16)
16	8.294722600	127.0.0.1	127.0.0.1	64	ICMP	Echo (ping) reply id=0x0001, seq=355/25345, ttl=128 (request in 15)

Рис. 14 – инфа по третьему пакету

Результаты формирования сложных фильтров, анализа ARP и TCP: Первый фильтр и результат захвата (файл cap1.pcapng).

Анализ TCP-сеанса. Протокол TCP – протокол транспортного уровня, обеспечивающий надёжную передачу информации между приложениями взаимодействующих устройств.

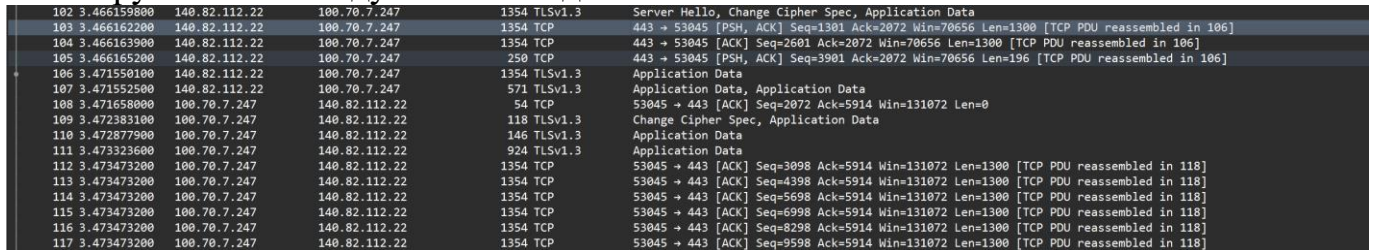
Первые пакеты (рукопожатие) – клиент предлагает начать соединение, указывает начальный Seq. Сервер согласен, подтверждает Seq клиента, шлет свой Seq. Клиент подтверждает Seq сервера. Соединение установлено.



No.	Time	Source	Destination	Length	Protocol	Info
34	3.205875400	100.70.7.247	140.82.112.22	66	TCP	53045 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
66	3.327568000	140.82.112.22	100.70.7.247	66	TCP	443 → 53045 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1300 SACK_PERM WS=1024
67	3.327785100	100.70.7.247	140.82.112.22	54	TCP	53045 → 443 [ACK] Seq=1 Ack=1 Win=131072 Len=0
75	3.342998000	100.70.7.247	140.82.112.22	1354	TCP	53045 → 443 [ACK] Seq=1 Ack=1 Win=131072 Len=1300 [TCP PDU reassembled in 76]

Рис. 19 – рукопожатие

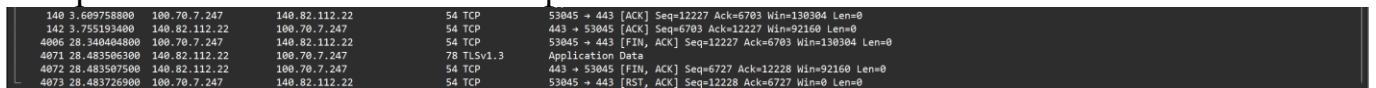
После рукопожатия идут пакеты с данными.



No.	Time	Source	Destination	Length	Protocol	Info
102	3.466159800	140.82.112.22	100.70.7.247	1354	TLSv1.3	Server Hello, Change Cipher Spec, Application Data
103	3.466162200	140.82.112.22	100.70.7.247	1354	TCP	443 → 53045 [PSH, ACK] Seq=1301 Ack=2072 Win=70656 Len=1300 [TCP PDU reassembled in 106]
104	3.466163900	140.82.112.22	100.70.7.247	1354	TCP	443 → 53045 [ACK] Seq=2601 Ack=2072 Win=70656 Len=1300 [TCP PDU reassembled in 106]
105	3.466165200	140.82.112.22	100.70.7.247	250	TCP	443 → 53045 [PSH, ACK] Seq=3901 Ack=2072 Win=70656 Len=196 [TCP PDU reassembled in 106]
106	3.471550100	140.82.112.22	100.70.7.247	1354	TLSv1.3	Application Data
107	3.471552500	140.82.112.22	100.70.7.247	571	TLSv1.3	Application Data, Application Data
108	3.471558000	100.70.7.247	140.82.112.22	54	TCP	53045 → 443 [ACK] Seq=2072 Ack=5914 Win=131072 Len=0
109	3.472383100	100.70.7.247	140.82.112.22	118	TLSv1.3	Change Cipher Spec, Application Data
110	3.472877900	100.70.7.247	140.82.112.22	146	TLSv1.3	Application Data
111	3.473323600	100.70.7.247	140.82.112.22	924	TLSv1.3	Application Data
112	3.473473200	100.70.7.247	140.82.112.22	1354	TCP	53045 → 443 [ACK] Seq=3098 Ack=5914 Win=131072 Len=1300 [TCP PDU reassembled in 118]
113	3.473473200	100.70.7.247	140.82.112.22	1354	TCP	53045 → 443 [ACK] Seq=4398 Ack=5914 Win=131072 Len=1300 [TCP PDU reassembled in 118]
114	3.473473200	100.70.7.247	140.82.112.22	1354	TCP	53045 → 443 [ACK] Seq=5698 Ack=5914 Win=131072 Len=1300 [TCP PDU reassembled in 118]
115	3.473473200	100.70.7.247	140.82.112.22	1354	TCP	53045 → 443 [ACK] Seq=6998 Ack=5914 Win=131072 Len=1300 [TCP PDU reassembled in 118]
116	3.473473200	100.70.7.247	140.82.112.22	1354	TCP	53045 → 443 [ACK] Seq=8298 Ack=5914 Win=131072 Len=1300 [TCP PDU reassembled in 118]
117	3.473473200	100.70.7.247	140.82.112.22	1354	TCP	53045 → 443 [ACK] Seq=9598 Ack=5914 Win=131072 Len=1300 [TCP PDU reassembled in 118]

Рис.20 – данные передаются

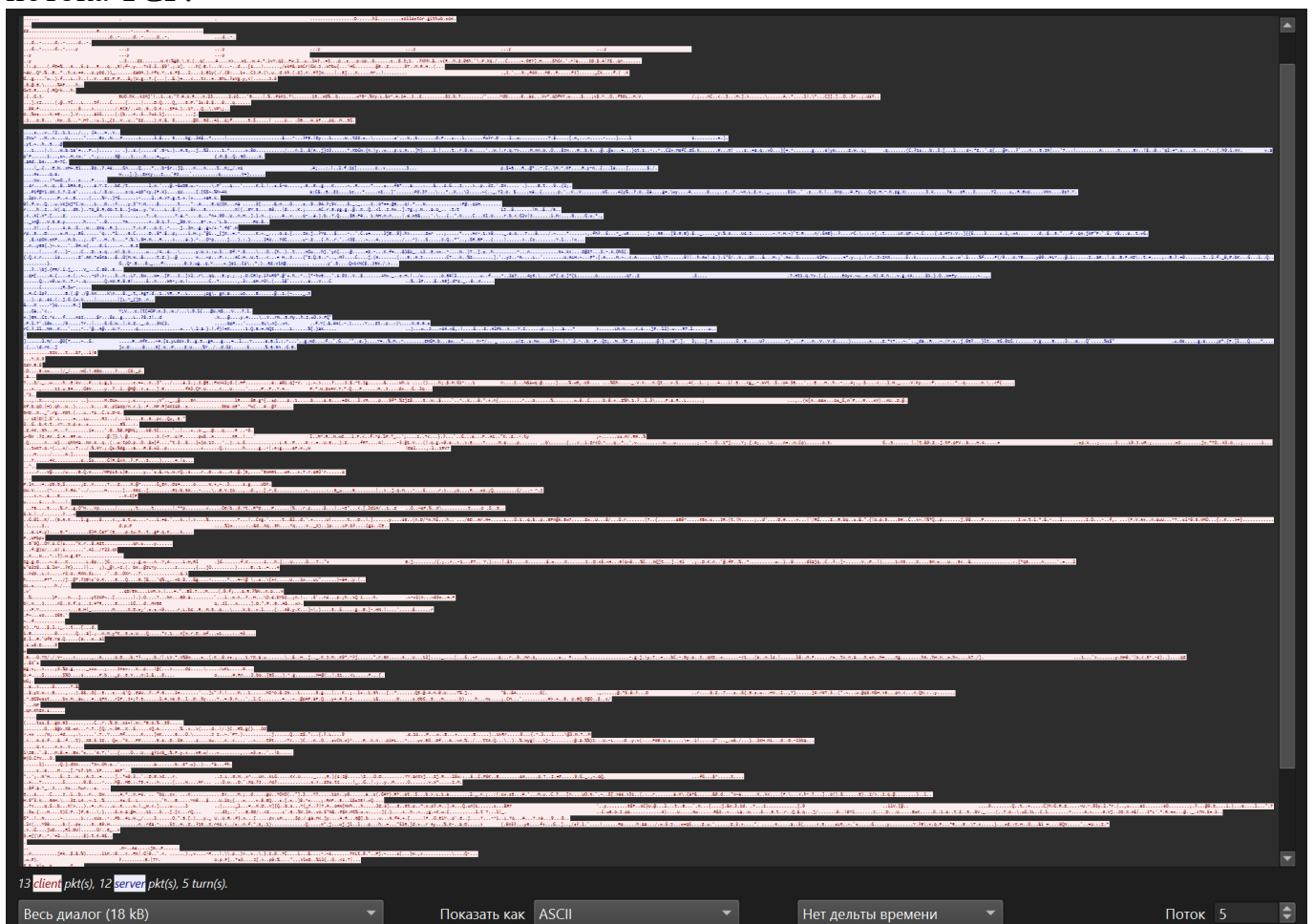
Завершение сеанса обозначается флагом FIN.



No.	Time	Source	Destination	Length	Protocol	Info
140	3.609758800	100.70.7.247	140.82.112.22	54	TCP	53045 → 443 [ACK] Seq=12227 Ack=6703 Win=130304 Len=0
142	3.755193400	140.82.112.22	100.70.7.247	54	TCP	443 → 53045 [ACK] Seq=6703 Ack=12227 Win=92160 Len=0
4806	28.340484800	100.70.7.247	140.82.112.22	54	TCP	53045 → 443 [FIN, ACK] Seq=12227 Ack=6703 Win=130304 Len=0
4871	28.483586300	140.82.112.22	100.70.7.247	78	TLSv1.3	Application Data
4872	28.483586300	140.82.112.22	100.70.7.247	54	TCP	443 → 53045 [FIN, ACK] Seq=6727 Ack=12228 Win=92160 Len=0
4873	28.483726900	100.70.7.247	140.82.112.22	54	TCP	53045 → 443 [RST, ACK] Seq=12228 Ack=6727 Win=0 Len=0

Рис.21 – завершение сеанса

Мы можем посмотреть диалог между клиентом и сервером через отслеживание потока TCP.



13 client pkt(s), 12 server pkt(s), 5 turn(s).

Весь диалог (18 kB) Показывать как ASCII Нет дельты времени Поток 5

Рис.22 – Весь диалог к-с (красный – от клиента, синий – от сервера)

Wireshark - Поток: Беспроводная сеть (tcp port 443)

Время	100.70.7.247	140.82.112.22	Комментарий
3.205875400	53045 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=	443	TCP: 53045 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460
3.327568000	443 → 53045 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 M=	443	TCP: 443 → 53045 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0
3.327785100	53045 → 443 [ACK] Seq=1 Ack=1 Win=131072 Len=0	443	TCP: 53045 → 443 [ACK] Seq=1 Ack=1 Win=131072 Len=0
3.342098000	53045 → 443 [ACK] Seq=1 Ack=1 Win=131072 Len=1300 [T=	443	TCP: 53045 → 443 [ACK] Seq=1 Ack=1 Win=131072 Len=1300
3.342098000	Client Hello (SNI=collector.github.com)	443	TLSv1.3: Client Hello (SNI=collector.github.com)
3.464958200	TCP Dup ACK 664#1 443 → 53045 [ACK] Seq=1 Ack=1 Win=	443	TCP: [TCP Dup ACK 664#1] 443 → 53045 [ACK] Seq=1 Ack=1 W=
3.464959700	443 → 53045 [ACK] Seq=1 Ack=2072 Win=70656 Len=0	443	TCP: 443 → 53045 [ACK] Seq=1 Ack=2072 Win=70656 Len=0
3.466159800	Server Hello, Change Cipher Spec, Application Data	443	TLSv1.3: Server Hello, Change Cipher Spec, Application Data
3.466162200	53045 → 53045 [PSH, ACK] Seq=1301 Ack=2072 Win=70656 Len=	443	TCP: 443 → 53045 [PSH, ACK] Seq=1301 Ack=2072 Win=70656
3.466163900	443 → 53045 [ACK] Seq=2601 Ack=2072 Win=70656 Len=1	443	TCP: 443 → 53045 [ACK] Seq=2601 Ack=2072 Win=70656 Len=1
3.466165200	443 → 53045 [PSH, ACK] Seq=3901 Ack=2072 Win=70656 Len=	443	TCP: 443 → 53045 [PSH, ACK] Seq=3901 Ack=2072 Win=70656
3.471550100	Application Data	443	TLSv1.3: Application Data
3.471552500	Application Data, Application Data	443	TLSv1.3: Application Data, Application Data
3.471658000	53045 → 443 [ACK] Seq=2072 Ack=5914 Win=131072 Len=0	443	TCP: 53045 → 443 [ACK] Seq=2072 Ack=5914 Win=131072 Le
3.472383100	Change Cipher Spec, Application Data	443	TLSv1.3: Change Cipher Spec, Application Data
3.472877900	Application Data	443	TLSv1.3: Application Data
3.473323600	Application Data	443	TLSv1.3: Application Data
3.473473200	53045 → 443 [ACK] Seq=3098 Ack=5914 Win=131072 Len=0	443	TCP: 53045 → 443 [ACK] Seq=3098 Ack=5914 Win=131072 Le
3.473473200	53045 → 443 [ACK] Seq=4398 Ack=5914 Win=131072 Len=0	443	TCP: 53045 → 443 [ACK] Seq=4398 Ack=5914 Win=131072 Le
3.473473200	53045 → 443 [ACK] Seq=5698 Ack=5914 Win=131072 Len=0	443	TCP: 53045 → 443 [ACK] Seq=5698 Ack=5914 Win=131072 Le
3.473473200	53045 → 443 [ACK] Seq=6998 Ack=5914 Win=131072 Len=0	443	TCP: 53045 → 443 [ACK] Seq=6998 Ack=5914 Win=131072 Le
3.473473200	53045 → 443 [ACK] Seq=8298 Ack=5914 Win=131072 Len=0	443	TCP: 53045 → 443 [ACK] Seq=8298 Ack=5914 Win=131072 Le

Packet 117: TCP: 53045 → 443 [ACK] Seq=9098 Ack=5914 Win=131072 Len=1300 TCP RST, reassembled in 118

Вывод: в ходе лабораторной работы был произведен анализ сетевого трафика компьютерных сетей на примере сайта gia.ru. Так же были изучены структуры сетевых протоколов различных уровней.